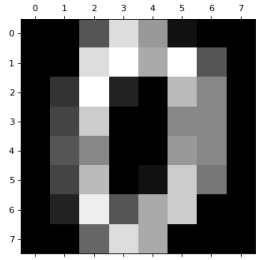
Data for Machine Learning

In Machine Learning (ML), a dataset consists of multiple **instances**. Each instance contains a **set of features (X)** and a corresponding **label (y)**.
The goal of a machine learning algorithm is to learn the relationship between the features and the labels to make predictions or classifications for new instances.

I. digits Dataset

This real dataset consists of 1797 images of size 8×8 pixels. Each image represents a handwritten digit ranging from 0 to 9, similar to the example shown below.



To use an image in a ML algorithm, it must first be transformed into a feature vector. Since each image has a size of 8×8 pixels, it contains: 8×8=64 pixels.
Each pixel is treated as a feature, resulting in a feature vector of length 64 for every image. The pixel values represent gray-level intensities, typically ranging from 0 to 16:

0 : white pixel
16 : very dark pixel

The goal of the learning algorithm is to classify each image into the correct digit category. Below are the main characteristics of this dataset:

Total number of instances: 1797
Number of classes: 10 (digits from 0 to 9)
Number of features: 64

Using the digits dataset with python

```
from sklearn.datasets import load_digits

# Load the dataset
digits = load_digits()

# Attributs/Features
X = digits.data

# Classes/Labels
y = digits.target
```

II. Synthetic Dataset

Synthetic classification datasets are artificially generated datasets created to simulate classification problems. Each dataset contains a set of instances, where every instance is described by features (X) and associated with a class label (y). These datasets allow researchers to control the properties of the data, such as the number of samples, number of features, class distribution, noise level, and the proportion of informative or redundant features. Synthetic datasets are widely used to test, evaluate, and compare machine learning algorithms under controlled experimental conditions.

The function **`make_classification`** from **`sklearn.datasets`** is used to generate synthetic classification datasets. It creates a set of instances where each instance has features (X) and a class label (y). Users can control the number of features, informative features, redundant features, repeated features, classes, and samples to simulate realistic classification problems for testing and experimentation.

Using the Synthetic dataset with python

```
from   sklearn.datasets import make_classification

X, y = make_classification( n_samples=1000,      # number of instances
                            n_features=50,       # total number of features
                            n_informative=5,     # features that are truly useful for classification
                            n_redundant=10,      # features correlated with the informative ones
                                                 # remaining features are irrelevant
                            random_state=42  )   # ensures the same dataset at each execution
```

KNN algorithm

The K-Nearest Neighbors (KNN) algorithm is a supervised machine learning method used for classification and regression. It works by finding the K closest instances (neighbors) to a new data point based on a distance metric (commonly Euclidean distance). For classification, the new instance is assigned the most frequent class among its K nearest neighbors.

Pseudo-code of K-Nearest Neighbors (KNN)

Input:
Training set $\boldsymbol{A} = \{(\boldsymbol{x}^{(1)},\ \boldsymbol{y}^{(1)}),\ (\boldsymbol{x}^{(2)},\ \boldsymbol{y}^{(2)}), \dots, (\boldsymbol{x}^{(n)},\ \boldsymbol{y}^{(n)})\}$
Instance to classify $\boldsymbol{x}$
Number of neighbors $\boldsymbol{K}$

Output:
Predicted class $\hat{\boldsymbol{y}}$

Procedure:
For each instance $(\boldsymbol{x}^{(i)},\ \boldsymbol{y}^{(i)}) \in \boldsymbol{A}$
Compute the Euclidean distance:

$$\boldsymbol{d}(\boldsymbol{x}, \boldsymbol{x}^{(i)}) = \sqrt{\sum_{j=1}^{d} \left(\boldsymbol{x}_j - \boldsymbol{x}_j^{(i)}\right)^2}$$

Sort the instances in $\boldsymbol{A}$ by increasing distance.
Select the $\boldsymbol{K}$ nearest neighbors.
Perform a majority vote on their classes.
Assign to $\boldsymbol{x}$ the most frequent class among its neighbors.
Return the predicted class $\hat{\boldsymbol{y}}$

Using KNN with python

```
from   sklearn.model_selection  import train_test_split
from   sklearn.neighbors        import KNeighborsClassifier
from   sklearn.metrics          import accuracy_score

# Split data into training and testing sets
# test_size=0.3 -> 30% of the data used for testing
# random_state=42 -> ensures the same split at each execution
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create the KNN model
KNN = KNeighborsClassifier(n_neighbors=5)

# Fit the model (store training data and prepare neighbor search)
KNN.fit(X_train, y_train)

# Make predictions
y_pred = KNN.predict(X_test)

# Evaluate model performance
accuracy = accuracy_score(y_test, y_pred)
```

Feature Selection

Feature selection is the process of selecting a subset of relevant features from the original dataset to build a machine learning model. The goal is to improve model performance, reduce computational cost, and eliminate irrelevant or redundant features while keeping the most informative ones.

I. Feature Selection with PSO

Solution Representation

A solution $\boldsymbol{x}$ in this problem is represented by a real-valued vector of dimension $\boldsymbol{D}$, where $\boldsymbol{D}$ is the number of features in the dataset. Each element $\boldsymbol{x_i} \in [\mathbf{0}, \mathbf{1}]$ represents the importance weight of feature $\boldsymbol{i}$.

Objective Function

The objective function is defined as:

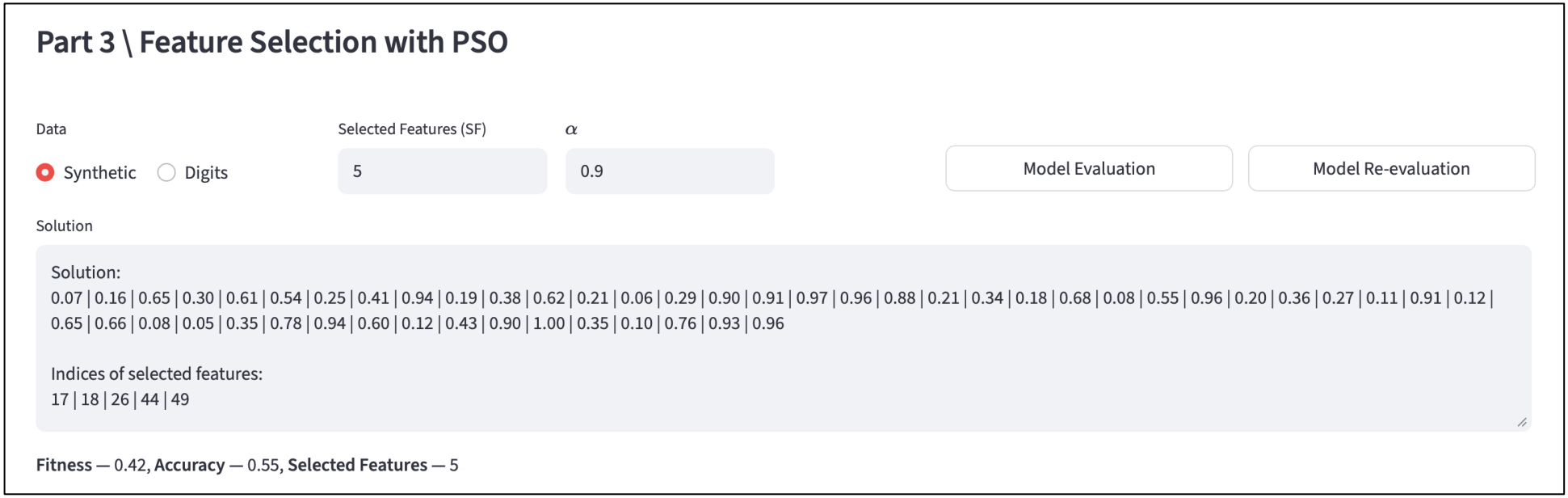
$$\boldsymbol{f(x) = \alpha * f_1(x) + (1 - \alpha) * f_2(x)}$$

Where:

$\boldsymbol{f_1(x) = 1 - \mathrm{Accuracy}}$, represents the classification error.

$\boldsymbol{f_2(x) = SF/D}$, $\boldsymbol{SF}$ is the number of selected features and $\boldsymbol{D}$ is the total number of features.

The selected features correspond to the $\boldsymbol{SF}$ features with the highest values of $\boldsymbol{x_i}$.



A tester :

Case 1 – Selected 25, Alpha 0.9

Solution:

0.74 | 0.56 | 0.79 | 0.92 | 0.28 | 0.13 | 0.53 | 0.80 | 0.49 | 0.91 | 0.91 | 0.88 | 0.71 | 0.96 | 0.31 | 0.30 | 0.01 | 0.14 | 0.36 | 0.42 | 0.53 | 0.99 | 0.73 | 0.53 | 0.84 | 0.10 | 0.34 | 0.63 | 0.02 | 0.29 | 0.46 | 0.30 | 0.18 | 0.21 | 0.23 | 0.78 | 0.59 | 0.50 | 0.27 | 0.30 | 0.36 | 0.99 | 0.15 | 0.60 | 0.03 | 0.37 | 0.52 | 0.12 | 0.32 | 0.69 | 0.48 | 0.91 | 0.45 | 0.57 | 0.46 | 0.62 | 0.68 | 0.48 | 0.27 | 0.94 | 0.47 | 0.70 | 0.12 | 0.35

Indices of selected features:

0 | 1 | 2 | 3 | 7 | 9 | 10 | 11 | 12 | 13 | 21 | 22 | 24 | 27 | 35 | 36 | 41 | 43 | 49 | 51 | 53 | 55 | 56 | 59 | 61

Case 2 – Selected 10, Alpha 0.9

Solution:

0.80 | 0.70 | 0.89 | 0.55 | 0.78 | 0.63 | 0.36 | 0.83 | 0.18 | 0.94 | 0.31 | 0.22 | 0.53 | 0.69 | 0.41 | 0.52 | 0.55 | 0.23 | 0.74 | 0.73 | 0.82 | 0.45 | 0.35 | 0.67 | 0.12 | 0.62 | 0.38 | 0.93 | 0.04 | 0.54 | 0.72 | 0.09 | 0.23 | 0.36 | 0.21 | 0.56 | 0.07 | 0.37 | 0.60 | 0.31 | 0.73 | 0.24 | 0.71 | 0.46 | 0.94 | 0.17 | 0.00 | 0.65 | 0.48 | 0.19 | 0.34 | 0.15 | 0.42 | 0.52 | 0.31 | 0.29 | 0.34 | 0.99 | 0.59 | 0.76 | 0.32 | 0.55 | 0.16 | 0.39

Indices of selected features:

0 | 2 | 4 | 7 | 9 | 20 | 27 | 44 | 57 | 59